The Phoenix Protocol

Picture this.

It is 3 a.m. on the eighth day of a nine-day simulation. A researcher named Rintaro has spent more than a week watching a progress bar slowly approach the finish line. Then, without warning, the power flickers for half a second.

Everything stops. Eight days of computation are lost, and he has to begin again from scratch.

Now imagine that, at any point, he could have pressed a button and saved the program's exact state. After the blackout, a single command would restore it to where it left off, as though the interruption had never happened.

That is the tool we want you to build. It has real-world applications: cloud providers use similar techniques to move running programs between machines, while serverless platforms rely on them to start programs within milliseconds. For the next few days, this is your challenge.

What you'll build

You will build two core capabilities, along with the programs needed to test them:

- A way to pause a running program and save everything needed to preserve its state.
- A way to restore that program so it can continue from the exact point at which it stopped.
- Your own test programs. For example, you might write a counter that prints numbers, a program that fills a large table with data, or one that writes lines to a file. You will use these programs to demonstrate that your tool works.

The five parts

Part 1: Save yourself

Write a program that saves its own state to a file. Stop the program, start it again, and make sure it continues from where it left off. For this part, the program is allowed to cooperate with your solution.

Part 2: Save points, like in a game

Use `fork()` to take a snapshot of a running program, then restore that snapshot whenever you choose. For example, pressing a key could make the program jump back to the state it was in ten seconds earlier.

Part 3: Crash-proof

We will terminate your program using `kill -9` at a random moment. When it starts again, it must have lost no more than one second of progress. Its save file must also never be left partially written or corrupted. This part will test how carefully and reliably you have engineered your solution.

The Phoenix Protocol

The five parts (continued)

Part 4: Freeze a stranger

This is the most challenging part. You must freeze a program that you did not write, working entirely from the outside and using only its process ID. You will need to pause the process, copy its memory and registers, and later create a new process that resumes execution from the same instruction. Start with a simple, single-threaded program.

Part 5: Go wild (pick any)

Choose any one of the following extensions:

- Freeze a program that uses multiple threads.
- Take a snapshot while the program continues running, limiting the pause to just a few milliseconds.
- Restore one saved image as ten copies at the same time.
- Restore a saved image on a different machine.

What to hand in

- A GitHub repository with the build that can be built with a single `make` command.
- A short README containing the exact commands needed to run each completed part.
- A `demo.sh` script for every part you implement. Each script should show the full process: starting the program, freezing or terminating it, restoring it, and demonstrating that it continues correctly.
- The source code for all your test programs.
- A short screen recording (YouTube Link), no longer than three minutes, demonstrating your best result.
- A one-page note describing what you attempted, what did not work, and what you learned.

How we'll mark it

We will build your project on a clean Ubuntu machine and run the demos you provide. For Part 4, we will also test your tool on a small program of our own, so your implementation must not be hardcoded to work only with your test programs.

Our first priority is whether each part works as intended: the restored program must have the correct state, with no progress lost or duplicated. We will then consider how robust and well-structured your solution is. Your write-up also matters; clear reasoning and an honest account of your results count, even if some parts are incomplete.

House rules

- Use C or C++ only, on Linux x86-64.
- You may use any system calls you need, including `fork`, `ptrace`, `/proc`, and `mmap`.

The Phoenix Protocol

- Do not use existing checkpointing tools such as CRIU or DMTCP. The goal is to build the functionality yourself.
- For Parts 1 to 4, keep your programs single-threaded and free of network communication. Anything beyond these constraints is a bonus.
- Be honest when evaluating your implementation. Do not claim that something works unless you have verified it. We will test everything.

Good luck. Bring a program back from the dead.